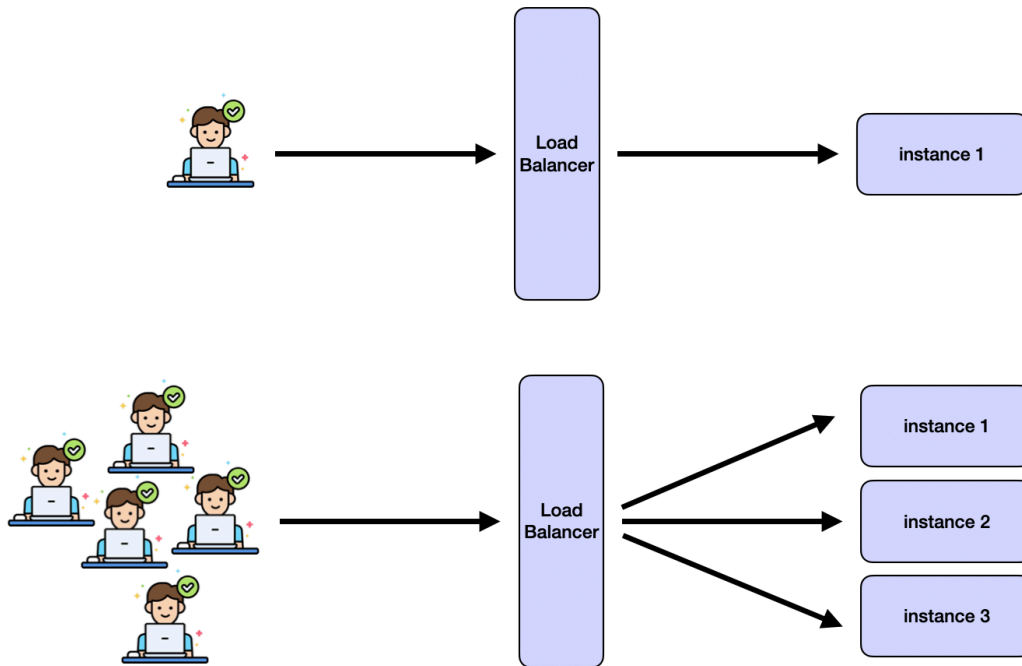


Auto Scaling

 systemdesignschool.io/fundamentals/auto-scaling



In the previous sections, we've talked about how to distribute traffic through load balancers and how load balancers can help distribute traffic to multiple instances using pre-defined rules. However, **what if the traffic is bursty or uneven through time?** Imagine a social media platform that experiences a sudden spike in traffic during a popular event such as a sports game or an e-commerce website that experiences a sudden spike in traffic during a black friday sale. We could always have a fixed number of instances to handle the maximum traffic possible, but that would be wasteful. What we can do instead is to have a system that can **automatically scale the number of instances up and down** based on the current traffic or the predicted traffic. This is where auto scaling comes in.

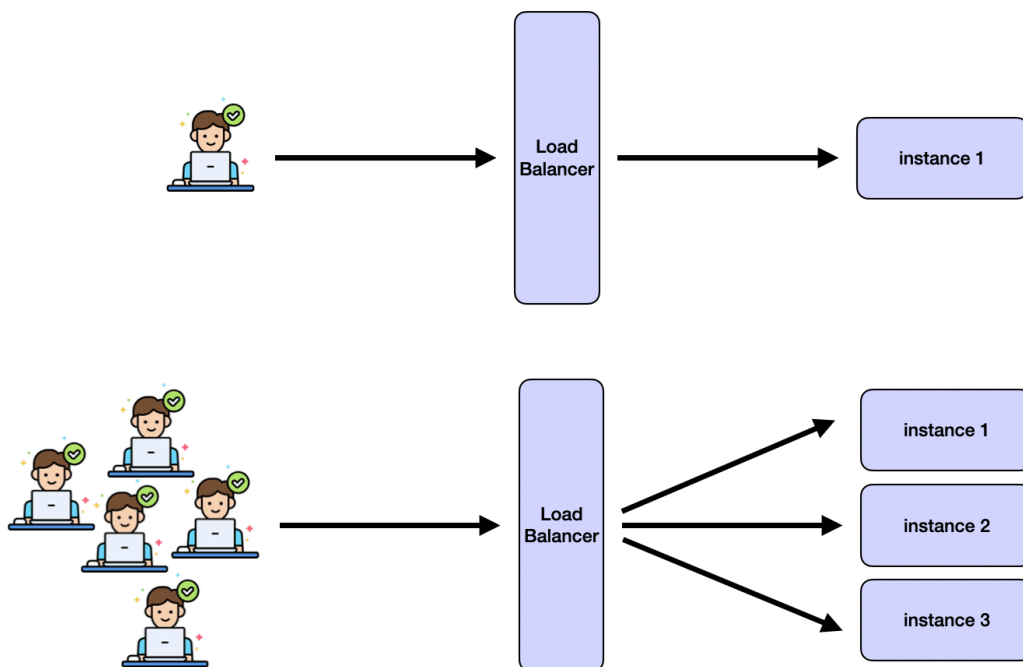
Auto Scaling is a cloud computing feature that automatically adjusts the number of computing resources (typically virtual machines or containers) based on the current demand. It works hand in hand with load balancers to ensure your application remains responsive and cost-efficient.

How Auto Scaling Works

The auto-scaling process begins with continuous monitoring of your application's metrics, including CPU utilization, memory usage, network traffic, request count, and any custom metrics you define. Based on predefined rules or policies, the system makes decisions about when to scale resources up or down.

When demand increases, the system initiates a scale-out (or scale-up) action to add more resources. Conversely, when demand decreases, it performs a scale-in (or scale-down) action to remove unnecessary resources. These scaling actions can take two primary forms:

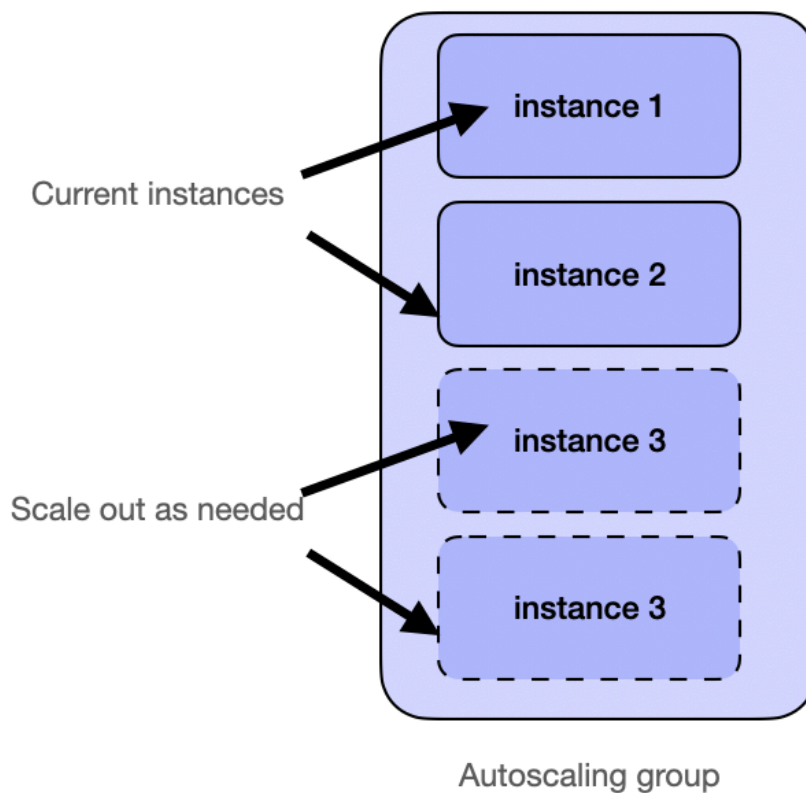
Horizontal scaling (Scale Out/In) involves adding or removing instances of resources, such as servers. This is the most common form of auto-scaling and is particularly well-suited for distributed applications. A typical example would be adding more web servers to handle increased traffic.



Auto Scaling Groups (ASG)

An Auto Scaling Group is a logical grouping of instances that share similar characteristics. It maintains a specified number of instances even if an instance becomes unhealthy. Here's how it works:

1. **Desired Capacity:** You set a desired number of instances that should be running
2. **Minimum/Maximum Limits:** You define boundaries for how small or large the group can scale
3. **Health Checks:** ASG continuously monitors instance health and automatically replaces failed instances
4. **Scaling Policies:** Define conditions for scaling (e.g., "add instance when CPU > 70%")
5. **Launch Template:** Specifies the instance configuration, i.e. what type of instance to launch



Auto Scaling Policies

Auto scaling can be implemented through two main approaches: reactive and predictive scaling. Reactive scaling responds to current demand and scales based on real-time metrics, making it ideal for unpredictable workloads. Predictive scaling, on the other hand, uses machine learning to forecast demand and scales preemptively, making it more suitable for workloads with predictable patterns, such as daily or weekly cycles.

A typical interview question is how to handle peak traffic during a black friday sale. The answer is to use predictive scaling to scale out the number of instances before the sale starts and scale in the number of instances after the sale ends. We can use previous year's data to predict the traffic during the black friday sale.

Benefits of Auto Scaling

The implementation of auto scaling brings several key advantages to your infrastructure. From a cost perspective, you only pay for the resources you actually need, with automatic capacity reduction during low-demand periods. High availability is enhanced as the system maintains performance during traffic spikes and automatically replaces unhealthy instances.

Users benefit from a consistent application experience with reduced latency, even during high-demand periods. Additionally, the automated nature of resource management significantly reduces the operational overhead and need for manual intervention.

Best Practices

When setting up your scaling rules, choose metrics that truly reflect your application's health and avoid configuring too-frequent scaling actions. Implement appropriate cooldown periods between scaling actions to prevent rapid oscillation between scaling up and down.

It's crucial to thoroughly test your scaling policies by simulating different load scenarios and verifying that the scaling behavior works as expected. Regular monitoring of costs through billing alerts and periodic review of scaling patterns helps maintain optimal resource utilization.

Integration with Load Balancers

Auto scaling and load balancers work in tandem to provide a comprehensive scaling solution. When new instances are added, the auto scaling system automatically registers them with the load balancer, which then begins distributing traffic to these new instances. During instance removal, the system ensures connections are gracefully drained before removing instances from the load balancer.

Other Use Cases

While compute instances (VMs/containers) are the most common resources to auto-scale, modern cloud platforms have evolved to support auto-scaling across many different components of your infrastructure.

Database resources can automatically scale to handle varying workloads. This includes dynamically adding or removing read replicas to distribute query load, adjusting storage capacity as data grows, scaling connection pools based on demand, and modifying query processing capacity to maintain performance.

Network resources aren't left behind – they can scale automatically too. Load balancers can adjust their capacity based on traffic patterns, while network bandwidth and connection limits can flex to accommodate changing demands. API gateways can dynamically adjust their rate limits based on usage patterns and service capacity.

Serverless components are inherently scalable, with function concurrency adjusting automatically to handle incoming requests. Event processing systems can scale their processing capacity based on queue depth, and worker pools can grow or shrink based on workload.

Cloud Provider Implementation

Major cloud providers have their own implementations of auto scaling services. AWS offers Auto Scaling Groups, Azure provides Virtual Machine Scale Sets, and Google Cloud implements Managed Instance Groups. While each platform offers similar core functionality, they differentiate themselves through platform-specific features and integrations.